Tensor Omics: Team Coding Guidelines

August 14, 2026

Contents

1 Compiler: Always Use the Latest gfortran

Always use `gfortran` 15 or newer. Newer compiler versions bring improved standard compliance, better auto-vectorization, and improved `do concurrent` support.

1.1 Using Docker (Recommended)

The project provides a pre-built Docker image with gfortran 15 and all required dependencies: no system-level changes are needed. Install Docker as explained for your operating system in the Docker documentation.

Step 1: Build the image from the `Dockerfile` in `misc/`:

```
cd misc
docker build -t arch-gfortran -f gfortran.docker .
cd ..
```

Step 2: Compile the project inside the container:

```
docker run -it -v $(pwd):/opt -w /opt arch-gfortran ./build.sh
```

1.2 Native Compilation

If you already have `gfortran` $\geq$ 15 installed, compile directly with `build.sh`. It compiles all files in `src/`, places compiled objects under `build/<compiler>/`, and creates a symbolic link to the resulting shared library (`libtensor-omics.so`) so that Python and R always find the same path.

Arch Linux (rolling release — always has the latest GCC):

```
sudo pacman -S gcc-fortran
```

macOS (via Homebrew):

```
brew install gcc
```

On Ubuntu/Debian, gfortran 15 is not available in the standard repositories. Use the Docker path instead.

Default — gfortran without optimisation flags:

```
./build.sh
```

Maximum performance — gfortran with optimisation flags:

```
./build.sh --max-performance
```

Intel Fortran Compiler — with maximum performance flags:

```
./build.sh --max-performance FC=ifx
```

1.3 Fortran Standard Reference

When in doubt about language semantics, consult the official Fortran standard draft:

https://j3-fortran.org/doc/year/26/26-007.pdf

2 File Extensions: `.F90` vs `.f90`

Fortran source files in this project use *two* extensions with a precise meaning:

- `.F90` (uppercase F): The file is passed through the C preprocessor before compilation. Use this extension whenever the file contains `#include`, `#define`, or any other preprocessor directive (e.g., `#include "macros.h"`).

- `.f90` (lowercase f): Plain Fortran, *no* preprocessor. Use this for files that contain no macros or includes.

Rule: If in doubt, use `.F90`. Do not mix files that use macros with the `.f90` extension, as gfortran will silently skip the preprocessor pass and produce cryptic compilation errors.

3 Code Formatting

3.1 Indentation

Use **4 spaces** per indentation level. Never use tabs. The project formatter is `fprettify`; run it before committing:

```
fprettify -i 4 -l 1000 <path-to-file>
```

The `-l 1000` flag sets a generous line-length limit so that `fprettify` does not introduce spurious line continuations in long argument lists.

3.2 One Dummy Argument Declaration Per Line

Each dummy argument (subroutine/function parameter) must be declared on its own line. This is required so that every argument can carry its own FORD documentation comment:

```
! CORRECT
integer(int32), intent(in) :: n_genes
  !! Total number of genes
real(real64), intent(in) :: expression_vectors(n_axes, n_genes)
  !! Gene expression matrix (n_axes x n_genes)

! WRONG: two arguments on one line cannot be individually documented
integer(int32), intent(in) :: n_genes, n_axes
```

For **local variables** this rule does not apply — grouping related locals on one line is acceptable. Local variables do not need FORD comments, but must still follow the naming conventions (meaningful, descriptive names).

4 Always Use Double Precision

All real variables and constants must use the ISO Fortran double precision kind `real64` from `iso_fortran_env`. Never use the default `real` kind or literal constants without a kind suffix.

```
use, intrinsic :: iso_fortran_env, only: real64, int32

real(real64) :: x, y
integer(int32) :: n

! Compile-time constants MUST carry the _real64 suffix
x = 3.141592653589793_real64
y = 2.718281828459045_real64
```

Omitting the `_real64` suffix causes the literal to be evaluated as single precision before being promoted, silently losing bits of accuracy.

5 Variable and Argument Naming Conventions

Consistent naming makes the code readable without comments. The following conventions are **mandatory**.

5.1 Prefixes and Suffixes

Pattern	Meaning	Example
`n_`	Size / count of a dimension	`n_genes`, `n_axes`, `n_families`
`i_`	Loop / iteration variable	`i_gene`, `i_axis`, `i_vec`
`_idx`	Calculated / derived index	`family_idx`, `out_idx`
`tmp_`	Work array (not input or output)	`tmp_perm`, `tmp_loess_y`
`_perm`	Permutation vector for `<name>`	`sort_perm`, `gene_perm`
`_impl`	Core implementation subroutine, and the module holding it	`k_means_assign_cluster_impl`
`_c`	C-interoperable wrapper (generated)	`compute_shift_vector_field_c`
`_expert`	Full-control wrapper with all work arrays exposed (generated)	`compute_family_scaling_expert`
`ierr`	Error code argument (always this name)	`integer(int32), intent(out) :: ierr`

5.2 Mapping Arrays

Use a descriptive `x_to_y` pattern for mapping arrays:

```
integer(int32), intent(in) :: gene_to_fam(n_genes)
! gene_to_fam(i_gene) = family index of gene i_gene

integer(int32) :: family_idx
family_idx = gene_to_fam(i_gene)
```

5.3 Boolean / Selection Arrays

Use the `_mask` or `_selection` suffix for logical index arrays:

```
logical, intent(in) :: vecs_selection_mask(n_vecs)
logical, intent(in) :: axes_selection(n_axes)
```

6 Routine Organisation: the Implementation / Entry Point / Expert Tiers

Every non-trivial computation is split into three tiers. This separation keeps the core logic pure and testable while giving callers a convenient entry point — and **you write only the first tier**. The other two are generated from it, so they cannot drift out of step with the algorithm or with each other.

6.1 Tier 1: `foo_impl` — Core Implementation (hand-written)

- Contains the actual algorithm, in a module whose name ends `_impl`.
- Accepts *everything* as explicit arguments: input arrays, output arrays, and any pre-allocated work arrays (`tmp_*`).
- Never allocates memory — not in the procedure, and not anywhere in an `_impl` module.
- Never validates input, and takes no `ierr` for it. It may still report a genuine *runtime* error (a division by zero, a failed external fit).
- **Must be `pure`** unless the algorithm genuinely cannot be. Purity is not annotated: the generator derives it, and emits pure wrappers wherever everything they call is pure.
- Every constraint the tiers above need is stated as an annotation on the argument it constrains — valid ranges (`DM_MIN`, `DM_MAX`), defaults (`DM_DEFAULT`), sizing (`DM_OUTPUT_FROM`). Finiteness is the default contract for every real argument; opt out with `DM_ALLOW_NAN` / `DM_ALLOW_INFINITE`.

```
pure subroutine quantile_normalization_impl(expr, n_genes, n_replicates, &
    normalized_expr, rank_means, tmp_col, tmp_perm)
  integer(int32), intent(in) :: n_genes
  integer(int32), intent(in) :: n_replicates
  real(real64), intent(in)  :: expr(n_replicates, n_genes)
  real(real64), intent(out) :: normalized_expr(n_replicates, n_genes)
  real(real64), intent(in)  :: rank_means(n_replicates)
  real(real64), intent(out) :: tmp_col(n_genes)      ! work array
  integer(int32), intent(out) :: tmp_perm(n_genes)   ! permutation work array
  ! ... pure implementation, no ierr, no validation, no allocation ...
end subroutine
```

6.2 Tier 2: `foo` — Validated Entry Point (generated)

- Validates every input, using the `tox_errors` validators, from the annotations on the implementation's arguments; returns early on failure with `ierr` set.
- Allocates the `tmp_*` work arrays with `M_ALLOCATE`, so they disappear from the signature the caller sees.
- Prepares whatever the algorithm assumes: builds and sorts a `<base>_perm` permutation, runs a `DM_PROLOGUE` if the implementation declares one.
- Takes `ierr` **last**, after the optional arguments.
- **This is the entry point to reach for first**, and the one the Python and R bindings publish.

6.3 Tier 3: `foo_expert` — Full Control (generated, when warranted)

- Validates exactly as `foo` does, then calls the implementation with what you supply: it allocates nothing and prepares nothing.
- Exists only where there is something to take over — your own permutation order, or a prologue you want to skip. Where `foo` does nothing beyond allocating, there is one wrapper and it is called `foo`. Roughly half the procedures are in that case.
- Published to Python and R only where the two tiers genuinely differ for those callers; Fortran and C always get both.

6.4 Why This Is Generated

The two wrapper tiers are almost entirely derivable from the implementation's signature and its argument documentation, and writing them by hand meant maintaining the same contract in three places at once. It also means the code generator now enforces the API: if a procedure's contract cannot be expressed in the annotation vocabulary, that is a sign the procedure is wrong rather than the generator. See `codegen_guide.md` for the annotation vocabulary, case by case.

7 Error Handling

All errors are communicated through a single integer output argument `ierr`. The `tox_errors` module defines all error codes and the functions to set and check them.

7.1 Error Code Categories

Range	Category	Example constant
0	Success	ERR_OK
1xx	I/O and file errors	ERR_FILE_OPEN, ERR_READ_DATA
2xx	Input validation	ERR_INVALID_INPUT, ERR_DIM_MISMATCH, ERR_NAN_INF
3xx	Memory	ERR_ALLOC_FAIL, ERR_POINTER_NULL
5xxx	Fortran runtime	ERR_UNIT_NOT_CONNECTED
9xxx	Internal / unknown	ERR_INTERNAL, ERR_UNKNOWN

7.2 Always Initialise `ierr` First

Call `set_ok(ierr)` at the start of every subroutine that uses `ierr`:

```
use tox_errors, only: set_ok, set_err, set_err_once, is_err, ERR_INVALID_INPUT

call set_ok(ierr)   ! ierr = ERR_OK = 0
```

7.3 Return Early on Error

Check `ierr` after every operation that can fail, and return immediately:

```
call validate_dimension_size(n_genes, ierr)
call validate_all_in_range_real(data, n_genes, ierr)
if (is_err(ierr)) return
```

7.4 Reporting the Failing Argument (`arg_pos`)

Every `validate_*` routine, as well as `set_err`, accepts an optional `arg_pos` argument: the 1-based position of the offending argument in the public subroutine's own signature. Passing it lets callers across the language boundary (C, Python, R) report exactly which argument failed, instead of only an error code:

```
call validate_dimension_size(n_clusters, ierr, arg_pos=1_int32)
call validate_dimension_size(n_points, ierr, arg_pos=3_int32)
call validate_dimension_size(n_dims, ierr, arg_pos=4_int32)
if (n_clusters > n_points) call set_err_once(ierr, ERR_INVALID_INPUT, arg_pos=1
    _int32)

call validate_all_in_range_real(data_points, size(data_points, kind=int32), ierr,
    arg_pos=2_int32)
call validate_all_in_range_real(centroids, size(centroids, kind=int32), ierr,
    arg_pos=5_int32)
```

`arg_pos` must always match the argument's position in the public (non-`_impl`) subroutine that callers actually invoke, never its position in whichever internal implementation happens to run the validation. The generated wrappers get this right by construction, and clear any position an `ierr` carries out of an implementation, since that one numbers a different dummy list.

When the valid range of an argument is itself defined by `CM_` macros (Section 11.2), pass those macros as the bounds, so the reported range and the validated range can never disagree:

```
call validate_in_range_int(method, ierr, min=CM_METHOD_AVERAGE, max=CM_METHOD_WARD,
     arg_pos=7_int32)
```

7.5 `set_err` vs `set_err_once`

- `set_err(ierr, code)`: *Unconditionally* sets `ierr` to `code`, overwriting any error already stored. Use it only when `ierr` is known to be `ERR_OK` (e.g. the first check after `set_ok`), or when you deliberately want the new code to take precedence.
- `set_err_once(ierr, code)`: Sets `ierr` to `code` *only if `ierr` is currently `ERR_OK`*, preserving the first error encountered. This is the safe default when chaining multiple fallible calls, and is exactly what the `validate_*` routines use internally.

7.6 Never Silent Errors

Do not ignore `ierr`. If a callee sets an error, the caller must either propagate it or handle it explicitly. Every downstream caller that receives an `ierr` must check it.

7.7 Avoid GOTO

Never use `GOTO`. It is prohibited in all new code for the following reasons:

- It creates non-local control flow that is impossible to follow, making error paths untraceable.
- It defeats the structured error model built around `ierr` and early returns.
- It prevents the compiler from performing flow analysis, blocking optimizations and `pure` qualification.
- It is incompatible with `do concurrent`, which requires structured concurrency regions.

How to replace `GOTO` correctly:

Pattern to replace	Correct Fortran replacement
Forward jump to error label	`call set_err(ierr, ...); return`
Loop exit (`GOTO after loop`)	`exit` or `cycle` inside a `do` loop
Conditional skip block	`if (...) then ... end if`
Multi-level exit	Refactor into a subroutine with early `return`

```
! WRONG
if (n <= 0) goto 99
call compute(n)
99 continue

! CORRECT
if (n <= 0) then
  call set_err(ierr, ERR_INVALID_INPUT)
  return
end if
call compute(n)
```

8 Use of `intent` and `pure`

8.1 Mandatory `intent`

Every argument in every subroutine and function must carry an `intent` attribute:

- `intent(in)`: The routine only reads this argument.
- `intent(out)`: The routine completely overwrites this argument.
- `intent(inout)`: The routine both reads and modifies the argument, and the incoming value matters.

Omitting `intent` disables aliasing checks and is forbidden.

8.2 Mandatory `pure` for Core Routines

All *helper* and core subroutines must be declared `pure`. A `pure` procedure:

- Has no side effects (no I/O, no global state changes).
- Is eligible for auto-vectorization and threading by the compiler.
- Allows the compiler to schedule calls more aggressively.

Remove `pure` only when truly necessary (e.g. I/O, or an impure external library). Allocation is not a reason: `allocate(..., stat=)` is legal in a pure procedure, so the allocating tier is as eligible for `pure` as any other, and the generator marks a wrapper `pure` whenever everything it calls is.

8.3 Optional Arguments

Fortran does not support default values in argument lists. Use the `M_DEFAULT_VAL` macro to assign defaults after a `PRESENT` check:

```
#include "macros.h"
subroutine my_routine(data, n, span, degree)
  real(real64), intent(in), optional :: span
  integer(int32), intent(in), optional :: degree
  real(real64) :: actual_span
  integer(int32) :: actual_degree

  M_DEFAULT_VAL(span, actual_span, 0.7_real64)
  M_DEFAULT_VAL(degree, actual_degree, 2_int32)
```

Never use optional arguments in C wrappers; `bind(C)` is incompatible with them.

9 Parallelisation: `do concurrent`

Prefer `do concurrent` over plain `do` loops whenever the iterations are independent. This is the primary parallelisation mechanism in this project. Do not use OpenMP pragma loops — let the compiler exploit `do concurrent` through its own vectorisation and threading back-ends.

9.1 Locality Specifications

Every variable used inside a `do concurrent` loop **must** carry an explicit locality specification. Do not rely on the compiler's assumption — leaving it implicit makes the intent ambiguous and can produce incorrect results with aggressive optimisers.

Specification	Meaning
`shared(...vars)`	All iterations read/write the *same* instance of the variable. Use for arrays being filled element-wise or for read-only scalars.
`local(...vars)`	Each iteration gets its own *private* copy, uninitialised at entry. Use for loop-local temporaries.
`local_init(...vars)`	Like `local`, but each copy is initialised to the value the variable had before the loop.
`reduce(op:...vars)`	Each iteration contributes to a scalar accumulator via `op`. The final result is the reduction of all per-iteration contributions.

Supported `reduce` operators: `+`, `*`, `max`, `min`, `iand`, `ieor`, `ior`, `.and.`, `.or.`, `.eqv.`, `.neqv.`

```
! Element-wise: array filled in parallel, scalar read-only
do concurrent (i_dim = 1:n_dims) shared(vector, vector_norm)
  vector(i_dim) = vector(i_dim) / vector_norm
end do

! Reduction: accumulate a sum over independent iterations
means_sum = 0.0_real64
do concurrent (i_gene = 1:n_genes) shared(gene_means) reduce(+:means_sum)
  means_sum = means_sum + gene_means(i_gene)
end do

! Local temporary: each iteration owns its own scratch variable
do concurrent (i_vec = 1:n_vectors) shared(mat, norms) local(tmp_norm)
  tmp_norm = norm2(mat(:, i_vec))
  norms(i_vec) = tmp_norm
end do
```

When to use `do concurrent`:

- Element-wise array operations.
- Independent distance or dot-product computations over gene/axis indices.
- Any loop where iteration order does not affect correctness.

When NOT to use `do concurrent`:

- Loops with data-dependent control flow across iterations (e.g., early-exit on error found inside a loop — in that case, validate before the loop).
- Sequential algorithms like merge steps of merge-sort.

10 Memory Model: No Copies, No Allocations in Cores

10.1 No Allocations in Core Routines

Nothing in an `_impl` module may allocate memory — not the implementation, not its private helpers. Allocation happens exclusively in the generated entry point, and the generator enforces the rule (it also bounds what an `_impl` module may `use`, so the rule cannot be side-stepped by calling out to another module). This:

- Keeps core routines `pure` and deterministic.
- Allows callers to reuse pre-allocated work arrays across multiple calls, avoiding repeated `malloc`/`free` cycles over large datasets.

10.2 No Defensive Copies

Avoid creating copies of input arrays. For large omics datasets (tens of thousands of genes, hundreds of tissues), an unexpected copy easily exceeds available memory. Design algorithms to work on slices and views of the original data.

10.3 Logical Masks

A logical mask is a `logical` array with the same shape as the data it describes: `.true.` marks an element to include, `.false.` marks it to exclude. Passing a mask alongside the full array lets a routine include or exclude elements in place — with a plain `if (mask(...))` check — instead of first allocating a smaller array and copying the kept elements into it. This saves both the allocation and the copy, which matters at big scale.

Prefer adding a mask argument over asking the caller to pre-filter their data into a separate array. In `tox_relative_axis_plane_tools.F90`, `omics_vector_RAP_projection` takes the full `vecs` matrix together with a `vecs_selection_mask`/`axes_selection_mask`, and only writes the selected entries into the (exactly-sized) output — the input matrix itself is never copied or filtered:

```
logical, dimension(n_vecs), intent(in) :: vecs_selection_mask
    !! `.true.` for vectors where projection is to be computed
...
do i_vec = 1, n_vecs
    if (vecs_selection_mask(i_vec)) then
        ...
        if (axes_selection_mask(i_axis)) then
            projections(i_axis_proj, i_vec_proj) = vecs(i_axis, i_vec)
            ...
```

Similarly, in `tox_data_integration_jsd_impl.F90`, an optional `neighbor_mask` lets `build_residual_histograms_impl` skip specific neighbors while iterating the original, unfiltered array:

```
logical, dimension(n_neighbors, n_points), intent(in), optional :: neighbor_mask
    !! Optional mask to exclude specific neighbors (e.g. for family-wise analysis)
...
filter_neighbors = present(neighbor_mask)
...
if (filter_neighbors) then
    if (.not. neighbor_mask(i_neighbor, i_point)) cycle
end if
```

No subset of `neighborhood_residuals` is ever built — the mask only decides which iterations contribute.

When a smaller, compacted array genuinely has to be produced (e.g. to hand a reduced gene list downstream), use `count(mask)` to size the allocation exactly once, rather than over-allocating or growing it as elements are found, as in `tox_data_tools.F90`:

```
n_genes_kept = count(mask)
allocate(temp_gene_ids(n_genes_kept), stat=ios)
...
do i = 1, n_genes_total
    if (mask(i)) then
        temp_gene_ids(j) = gene_ids(i)
        ...
        j = j + 1
    end if
end do
```

Use masks wherever a routine only needs to act on part of its input: they are cheaper than copying, and they keep the calling convention uniform (same array, same shape, one extra `logical` argument) whether or not filtering is requested.

10.4 Work Arrays (`tmp_`)

Work arrays must be pre-allocated by the caller and passed as explicit arguments to core routines. They follow the `tmp_` prefix convention.

```
! tmp_perm holds the reusable sort permutation
integer(int32), intent(out) :: tmp_perm(n_genes)
real(real64),   intent(out) :: tmp_loess_y(n_genes)
```

10.5 Reusing Output Buffers as Work Arrays (Pointer Reshaping)

When a routine has already been handed a large enough output (or other pre-allocated) buffer, reuse it as scratch space via Fortran pointer bounds-remapping instead of allocating a separate temporary. This is a zero-copy *reshape* of contiguous storage: the pointer sees the same memory under new bounds.

```
! Reinterpret a contiguous buffer as a 2-D work view (no copy, no allocation)
real(real64), dimension(:, :), pointer :: work_view
work_view(1:n_genes, 1:n_tissues) => output_buffer
! A column slice can then serve as a work pointer
tmp_col_ptr => work_view(:, 1)
```

This reshapes; it does not transpose. Bounds-remapping reinterprets the linear, column-major storage under the new shape: `work_view(i, j)` is element `(j-1)*n_genes + i` of `output_buffer`, *not* `output_buffer(j, i)`. A genuine transpose reorders elements and therefore requires moving data (e.g. `transpose` into a separate buffer); a pointer view can never transpose. The target must be contiguous.

Only reuse a buffer as scratch when its final output value is not needed until after the scratch use has finished — otherwise the scratch writes corrupt the output.

10.6 Always Use Pointers, Never Value-Copy Scalars in C Wrappers

See Section 15.

11 Macro Usage (`src/macros.h`)

All source files that use macros must start with:

```
#include "macros.h"
```

11.1 Available Macros

Macro	Purpose
`M_USE_NULL_VALIDATION`	Imports modules needed for `c_loc`/null checks in C wrappers
`M_CHECK_IERR_NON_NULL`	Silently returns if `ierr` pointer is null (always first check)
`M_CHECK_NON_NULL(ARG)`	Sets `ierr` to `ERR_POINTER_NULL` and returns if `ARG` pointer is null
`M_DEFAULT_VAL(OPT,LOC,DEF)`	Assigns `OPT` to `LOC` if present, else assigns `DEF`
`M_ALLOCATE(DECL)`	Allocates `DECL`; sets `ERR_ALLOC_FAIL` and returns on failure
`M_NAN`	IEEE quiet NaN (double)
`M_NEG_INF`	IEEE negative infinity (double)
`M_POS_INF`	IEEE positive infinity (double)

11.2 Module-Local Constants via `#define` (`CM_` prefix)

Global, reusable macros live in `macros.h` with the `M_` prefix. A constant that only makes sense within a single module does not belong there — adding it would clutter a file meant for cross-module reuse. Instead, define it locally, immediately after the `#include` line, using the `CM_` prefix ("Custom Macro"), and keep it next to the module's other constants (its `parameter` declarations):

```
#define CM_METHOD_AVERAGE 0
#define CM_METHOD_WEIGHTED 1
#define CM_METHOD_WARD 2

integer(int32), parameter :: METHOD_AVERAGE  = CM_METHOD_AVERAGE
integer(int32), parameter :: METHOD_WEIGHTED = CM_METHOD_WEIGHTED
integer(int32), parameter :: METHOD_WARD     = CM_METHOD_WARD
```

Keeping FORD documentation in sync. FORD does not resolve Fortran `parameter` values, but it does run the C preprocessor before generating docs, so a `CM_` macro referenced inside a doc comment renders as-is. This lets a mode/method table stay anchored to the code instead of to a literal that can drift:

```
!! |       Method       |           Value          |
!! |--------------------|--------------------------|
!! | Average / UPGMA    |   CM_METHOD_AVERAGE      |
!! | Weighted / WPGMA   |   CM_METHOD_WEIGHTED     |
!! |       Ward         |   CM_METHOD_WARD         |
```

Use the macro in validation too. When a `CM_` macro defines the valid values or bounds of an argument, pass the macro itself to the corresponding `validate_*` call instead of repeating the literal:

```
call validate_in_range_int(method, ierr, min=CM_METHOD_AVERAGE, max=CM_METHOD_WARD,
    arg_pos=7_int32)
```

This keeps the default/bound, the validation, and the documentation all traceable to one definition — if a mode is added or reordered, only the `#define` block needs to change.

12 Derived Types (`type`): Use with Care

Derived types (`type :: ...`) are a powerful Fortran feature but must be used with restraint in this project:

- **Prohibited** in any subroutine or function that has a C wrapper or is otherwise called from Python or R. C, Python `ctypes`, and R's C interface cannot represent Fortran derived types; passing them across the language boundary is undefined behaviour.
- **Permitted** for pure-Fortran data structures where the extra abstraction genuinely improves readability or correctness (e.g., hash maps, JSON serialisation helpers).
- **Avoid by default** in the main TOX scientific routines, which operate exclusively on plain arrays and matrices.

If you need a compound structure that must cross the language boundary, decompose it into flat arrays and pass its components individually.

13 The `f42_utils` Utility Module

The file `src/f42_utils.F90` contains general-purpose helper functions shared across the project: `mean`, `std_dev`, `norm`, `clamp_real`/`clamp_int`, `sort_array`, `angle_between`, `is_close`, `logx`, `compute_edf`, etc.

Rule: Before implementing a new helper function anywhere else in the codebase, check whether `f42_utils` already provides it. If you write a function that is genuinely reusable across modules, add it to `f42_utils` rather than creating a module-local copy. This prevents duplication and keeps utility logic in one place.

```
use f42_utils, only: norm, is_close, logx, mean, std_dev, sort_array
```

14 The `safeguard` Module

Every module that defines C wrappers must `use safeguard`. This module contains compile-time assertions that verify the C and Fortran kind constants are equivalent on the target platform:

```
use safeguard   ! guarantees c_int == int32, c_double == real64, c_double_complex
    == real64
```

If these kinds do not match, the module fails to compile with a descriptive error, preventing silent ABI (Application Binary Interface) mismatch bugs. The ABI is the low-level contract between compiled code: it specifies how function arguments are laid out in memory, what sizes the types occupy, and how values are returned. When the Fortran kind for `real64` (8 bytes) does not match the C kind for `c_double` on a given platform, data passed through a C wrapper would be misinterpreted silently, producing incorrect results without any runtime error.

15 C Wrapper Conventions

C wrappers expose Fortran routines to C, Python, and R. They must follow a strict template to ensure ABI stability, null safety, and zero-copy operation.

15.1 Naming Conventions

- A standard wrapper for function `foo` is named `foo_c`.
- A wrapper that exposes all work arrays for fine-grained control is named `foo_expert_c`.
- The `bind(C, name="...")` string must match exactly the function name used in Python and R.

15.2 Complete Wrapper Template

```
#include "macros.h"

pure subroutine compute_tissue_versatility_c( &
    n_axes, n_vectors, expression_vectors,    &
    exp_vecs_selection_index,                 &
    n_selected_vectors, axes_selection,       &
    n_selected_axes,                          &
    tissue_versatilities, tissue_angles_deg,  &
    ierr) bind(C, name="compute_tissue_versatility_c")

  use, intrinsic :: iso_c_binding, only: c_int, c_double
  use tox_tissue_versatility, only: compute_tissue_versatility
  M_USE_NULL_VALIDATION
  implicit none

  ! All scalars and arrays must have the target attribute
  integer(c_int), intent(in),  target :: n_axes
    !! Number of axes (dimensions) - same doc as core routine
  integer(c_int), intent(in),  target :: n_vectors
    !! Total number of expression vectors
  integer(c_int), intent(in),  target :: n_selected_axes
    !! Number of selected axes (count of non-zero entries in axes_selection)
  integer(c_int), intent(in),  target :: n_selected_vectors
    !! Number of selected vectors (count of non-zero entries in
        exp_vecs_selection_index)
  real(c_double), intent(in),  target :: expression_vectors(n_axes, n_vectors)
    !! Gene expression matrix (n_axes x n_vectors), column-major
  integer(c_int), intent(in),  target :: exp_vecs_selection_index(n_vectors)
    !! Selection mask for vectors: 1 = include, 0 = skip (logical converted to int)
  integer(c_int), intent(in),  target :: axes_selection(n_axes)
    !! Selection mask for axes: 1 = include, 0 = skip (logical converted to int)
  real(c_double), intent(out), target :: tissue_versatilities(n_selected_vectors)
    !! Output: tissue versatility score for each selected vector
  real(c_double), intent(out), target :: tissue_angles_deg(n_selected_vectors)
    !! Output: angle in degrees for each selected vector
```

```
  integer(c_int), intent(out), target :: ierr
    !! Error code: 0 = success

  ! 1. Always check ierr pointer first (silent return if null)
  M_CHECK_IERR_NON_NULL
  ! 2. Check all other pointers
  M_CHECK_NON_NULL(n_axes)
  M_CHECK_NON_NULL(n_vectors)
  M_CHECK_NON_NULL(n_selected_axes)
  M_CHECK_NON_NULL(n_selected_vectors)
  M_CHECK_NON_NULL(expression_vectors)
  M_CHECK_NON_NULL(exp_vecs_selection_index)
  M_CHECK_NON_NULL(axes_selection)
  M_CHECK_NON_NULL(tissue_versatilities)
  M_CHECK_NON_NULL(tissue_angles_deg)

  ! 3. Delegate to the core routine
  call compute_tissue_versatility(n_axes, n_vectors, expression_vectors, &
      exp_vecs_selection_index, n_selected_vectors, axes_selection,       &
      n_selected_axes, tissue_versatilities, tissue_angles_deg, ierr)

end subroutine compute_tissue_versatility_c
```

15.3 Key Rules

1. **Use `iso_c_binding` types exclusively**: `real(c_double)`, `integer(c_int)`, `character(kind=c_char)`.
2. **All arguments must have `target`**: Required for `c_loc()` used by the null-check macros.
3. **Never use `value`**: All arguments are passed by reference. This is both the Fortran default and a requirement for zero-copy operation across the boundary.
4. **Always explicit array dimensions**: Do not use assumed-shape `(:)` in `bind(C)` routines.
5. **No computation in wrappers**: The wrapper only validates pointers and delegates to the core.
6. **No optional arguments**: `bind(C)` and optional arguments are incompatible. Override via the `_expert_c` variant.
7. **No derived types**: Pass components as flat scalars and arrays.
8. **Stable symbol names**: The `bind(C, name="...")` string is the ABI surface. Never rename it after publication.
9. **No aliasing**: Never pass the same memory region under two different argument names.
10. **Copy FORD comments from the core routine**: Every argument declaration in the C wrapper must carry the same `!!` documentation comment as the corresponding argument in the original core subroutine.

15.4 Character / String Arguments

When the Fortran core uses `character(len=*)`, the C wrapper receives a fixed-length character array plus an explicit length integer. Always use **named type parameters** for character declarations to avoid ambiguity between `len` and `kind`:

```
! CORRECT: named parameters, unambiguous
integer(c_int), intent(in), target :: filename_len
character(len=1, kind=c_char), dimension(filename_len), intent(in), target ::
    filename
```

```
! CORRECT: multi-dimensional string arrays (len=1 per character element, kind=
    c_char)
! One dummy argument per line (see Section 3); each can carry its own FORD comment
integer(c_int), intent(in), target :: max_str_len
integer(c_int), intent(in), target :: d1
integer(c_int), intent(in), target :: d2
integer(c_int), intent(in), target :: d3
character(len=1, kind=c_char), dimension(max_str_len, d1, d2, d3), intent(in),
    target :: strings_nd

! WRONG: positional parameter character(c_char) means len=c_char=1, kind=default
! This happens to work because c_char==1, but is semantically incorrect.
character(c_char), dimension(str_len) :: label   ! avoid this
```

The string length must always be passed as a separate integer argument; there is no mechanism to infer it from the C side. After receiving the array, convert it to a Fortran string before calling the core:

```
character(len=:), allocatable :: fortran_str
call c_char_1d_as_string(filename, filename_len, fortran_str, ierr)
if (is_err(ierr)) return
```

15.5 Logical Arguments

Fortran's `logical` type is **not interoperable with C** and must never appear in a `bind(C)` wrapper. Replace every `logical` argument with `integer(c_int)`. The convention is `0 = .false.`, **any non-zero = .true.**

The wrapper declares a local `logical` variable and converts via the project utility `c_int_as_logical` (from `tox_conversions`):

```
! Core routine uses logical
pure subroutine compute_tissue_versatility(..., axes_selection, ...)
  logical, intent(in) :: axes_selection(n_axes)
    !! Selection mask for axes: .true. = include, .false. = skip
  ...
end subroutine

! C wrapper receives integer(c_int), converts to logical before calling core
pure subroutine compute_tissue_versatility_c(..., axes_selection, ...) &
    bind(C, name="compute_tissue_versatility_c")
  use tox_conversions, only: c_int_as_logical
  integer(c_int), intent(in), target :: axes_selection(n_axes)
    !! Selection mask for axes: 1 = include, 0 = skip (any non-zero is .true.)
  logical :: axes_selection_f(n_axes)
  ...
  call c_int_as_logical(axes_selection, axes_selection_f)
  call compute_tissue_versatility(..., axes_selection_f, ...)
end subroutine
```

The reverse conversion (for `intent(out)` logical arguments) uses `logical_as_c_int`, also from `tox_conversions`: `0` for `.false.`, `1` for `.true.`

The generated bindings do this conversion for you: the Python wrapper passes a `numpy int32` array and the R wrapper an `as.integer()` vector.

15.6 When Not to Write a Wrapper

Only create C wrappers for functionality that is genuinely absent in Python and R. Do not wrap:

- Sorting routines (Python `numpy.sort`, R `order()` already exist).
- Generic I/O or string utilities.

15.7 Mandatory Development Workflow

When implementing new scientific functionality, complete these steps *in order* and include all of them in the same Merge Request:

1. **Core Fortran implementation.** Write the `foo_impl` pure subroutine, in a module whose name ends `_impl`. It takes no `ierr`, validates nothing and allocates nothing — it is the algorithm only. Everything the layers above it need, it states as annotations on its own arguments: valid ranges via `DM_MIN`/`DM_MAX`, work arrays via the `tmp_` prefix, sizing via `DM_OUTPUT_FROM`, and so on. See `codegen_guide.md` for the full vocabulary, case by case.

2. **Generate the layers.** Run `python helper/generate_code.py` (every `./build.sh` does this for you unless `--skip-code-generation` is passed). From the annotated implementation it writes, into `src/generated/`: the validating entry point `foo` — and `foo_expert` beside it where there is something for a caller to take over — the `bind(C)` wrapper, the Python `ctypes` module, the R wrapper with its pure-C `.Call` shim, and the VS Code snippets. **None of these is hand-written and none of them may be edited**; a change belongs in the implementation's source or its annotations.

3. **FORD documentation.** Ensure all public procedures have `!>` and `!!` comments. Documentation is built by CI, but verify the output locally for non-trivial API changes.

4. **Tests.** Write Fortran unit tests that cover numerical correctness. Add Python/R tests that verify the function is callable and returns the expected type and shape. AI-generated language-wrapper tests are acceptable when they avoid duplicating work at no loss of correctness coverage.

16 R / C `.Call` / Fortran Interface

The R interface is a four-layer stack, and the top two layers are **generated** — this section describes what comes out, so that you can read it, not so that you can write it.

1. **Fortran core** — pure computation, no language knowledge.

2. **C wrapper (`bind(C)`)** — type-safe bridge, pointer validation.

3. **C `.Call` shim** — allocates the R vectors, calls the `bind(C)` wrapper, and returns a named list. It is plain C against R's own API; the project does **not** use Rcpp, so no C++ toolchain is needed. The shims compile into `build/libtensor-omics.so` along with the rest of the library.

4. **R layer** — input validation and the user-facing API; reaches the shim by name through `.Call`.

16.1 What the Generated Shim Looks Like

```
/* generated; resolved by dynamic lookup and called from R as
   .Call("compute_tissue_versatility_call", ...) */
SEXP compute_tissue_versatility_call(SEXP expression_vectors,
                                     SEXP vectors_selection_mask,
                                     SEXP axes_selection_mask) {
    int n_axes    = Rf_nrows(expression_vectors);
    int n_vectors = Rf_ncols(expression_vectors);
    /* ... allocate the outputs, call compute_tissue_versatility_c, ... */
    /* ... assemble a named list of the outputs plus ierr ... */
}
```

16.2 What the Generated R Wrapper Looks Like

Input validation belongs to the R layer, not to the C shim. The generated wrapper calls the shared validators in `r/tensor_omics/tox_validate.R`, which the generator also writes:

```
compute_tissue_versatility <- function(expression_vectors,
                                       vectors_selection_mask,
                                       axes_selection_mask) {
  # validation derived from the implementation's argument annotations
  # ...
  result <- .Call("compute_tissue_versatility_call",
                  expression_vectors, vectors_selection_mask, axes_selection_mask)
  check_err_code(result$ierr)
  # ... return the outputs
}
```

16.3 Key Points

- **Do not edit `r/tensor_omics/`.** The whole directory is generated. The one hand-written file is `r/load_tensor_omics.R`, which loads the shared library and sources the generated wrappers.
- Roxygen documentation is generated too, from the Fortran doc comments. An implementation's `!>` and `!!` blocks *are* the R (and Python, and C) documentation, so write them as API prose for a reader of that language rather than as notes to yourself.
- `ierr` is always returned alongside the outputs and checked by the R wrapper via `check_err_code`. The argument position it carries names the argument the caller actually passed, not the derived one it was detected on.
- Column-major memory layout is the natural Fortran storage order; R matrices are column-major by default — no transposition is needed.
- **Memory ownership.** While R generally prevents external mutation of objects, some in-place data manipulation libraries (e.g. `data.table`) can modify underlying buffers. Do not pass aliases of live R objects to Fortran wrappers if those objects may be mutated elsewhere during the same analysis session.

17 Python / ctypes / Fortran Interface

The Python package `python/tensor_omics/` is **generated**, exactly as the R one is. What follows is the contract it implements — worth knowing because it is what your annotations buy, and because a caller has to respect it.

17.1 Loading the Shared Library

```
import ctypes, numpy as np, os
from error_handling import check_err_code

dll_path = os.path.abspath("build/libtensor-omics.so")
ctypes.CDLL("libgomp.so.1", mode=ctypes.RTLD_GLOBAL)
lib = ctypes.CDLL(dll_path)
```

17.2 Python Wrapper Pattern

```
#> tox_tissue_versatility:compute_tissue_versatility_c: Computes normalized tissue
    versatility for selected expression vectors
def tox_calculate_tissue_versatility(expression_vectors, vector_selection,
    axis_selection):
    """
    Calculate Tissue Versatility

    Args:
        expression_vectors: Matrix where each column is a gene expression vector (
            n_axes x n_vectors).
        vector_selection: Boolean or integer array indicating which vectors to
            process (length n_vectors).
        axis_selection: Boolean or integer array indicating which axes to include
            in calculation (length n_axes).

    Returns:
        dict: {
            'tissue_versatilities' (np.ndarray): Normalized tissue versatility
                values in [0,1] for selected vectors,
            'tissue_angles_deg' (np.ndarray): Angles in degrees [0,90] for selected
                vectors
        }
    """
    # Input validation
    if not isinstance(expression_vectors, np.ndarray):
        raise ValueError("expression_vectors must be a numpy array")

    if expression_vectors.ndim != 2:
        raise ValueError("expression_vectors must be a 2D array")

    # Convert inputs to numpy arrays
    expr_f = np.asfortranarray(expression_vectors, dtype=np.float64)
    select_vec = np.ascontiguousarray(vector_selection, dtype=bool).astype(np.int32
        )
    select_axes = np.ascontiguousarray(axis_selection, dtype=bool).astype(np.int32)

    # Get dimensions
    n_axes, n_vectors = expr_f.shape

    # Validate dimensions
    if len(select_vec) != n_vectors:
        raise ValueError("vector_selection length must match number of columns in
            expression_vectors")
    if len(select_axes) != n_axes:
        raise ValueError("axis_selection length must match number of rows in
            expression_vectors")

    # Calculate counts
    n_selected_vectors = np.sum(vector_selection)
    n_selected_axes = np.sum(axis_selection)

    # Convert scalar parameters
    n_axes_c = ctypes.c_int(n_axes)
    n_vectors_c = ctypes.c_int(n_vectors)
    n_selected_vectors_c = ctypes.c_int(n_selected_vectors)
    n_selected_axes_c = ctypes.c_int(n_selected_axes)
```

```
    # Prepare output arrays
    tissue_versatilities = np.empty(n_selected_vectors, dtype=np.float64)
    tissue_angles_deg = np.empty(n_selected_vectors, dtype=np.float64)
    ierr = ctypes.c_int(0)

    # Setup C/Fortran wrapper with proper type annotations
    tv = lib.compute_tissue_versatility_c
    tv.argtypes = [
        ctypes.POINTER(ctypes.c_int),  # n_axes
        ctypes.POINTER(ctypes.c_int),  # n_vectors
        np.ctypeslib.ndpointer(dtype=np.float64, flags="F_CONTIGUOUS"),  #
            expression_vectors (Fortran order)
        np.ctypeslib.ndpointer(dtype=np.int32, flags="C_CONTIGUOUS"),    #
            exp_vecs_selection_index
        ctypes.POINTER(ctypes.c_int),  # n_selected_vectors
        np.ctypeslib.ndpointer(dtype=np.int32, flags="C_CONTIGUOUS"),    #
            axes_selection
        ctypes.POINTER(ctypes.c_int),  # n_selected_axes
        np.ctypeslib.ndpointer(dtype=np.float64, flags="C_CONTIGUOUS"),  #
            tissue_versatilities
        np.ctypeslib.ndpointer(dtype=np.float64, flags="C_CONTIGUOUS"),  #
            tissue_angles_deg
        ctypes.POINTER(ctypes.c_int),  # ierr
    ]
    tv.restype = None

    # Call the C/Fortran wrapper
    tv(
        ctypes.byref(n_axes_c),
        ctypes.byref(n_vectors_c),
        expr_f,
        select_vec,
        ctypes.byref(n_selected_vectors_c),
        select_axes,
        ctypes.byref(n_selected_axes_c),
        tissue_versatilities,
        tissue_angles_deg,
        ctypes.byref(ierr)
    )

    # Check for errors and throw informative messages
    check_err_code(ierr.value)

    # Mark outputs as read-only
    _readonly(tissue_versatilities, tissue_angles_deg)

    # Return structured result (no ierr since we checked for errors)
    return {
        'tissue_versatilities': tissue_versatilities,
        'tissue_angles_deg': tissue_angles_deg
    }
```

17.3 Key Points

- Arrays with more than one dimension passed to Fortran must be **Fortran-contiguous**. Always create new arrays with `order='F'` (e.g. `np.empty(shape, order='F')`) so they are Fortran-contiguous from the start and no conversion copy is ever needed. Arrays coming from the caller must still be passed through `np.asfortranarray`: it only copies when the layout isn't already correct, so it stays cheap

while guaranteeing the right layout for input we don't control.

- Scalar parameters must be passed **by reference** using `ctypes.byref(c_int(value))`.
- All output arrays must be marked **read-only** after the call with `_readonly(...)`. This prevents accidental mutation of the output buffer by any subsequent Python operation or third-party library (e.g. `matplotlib`, `scipy`) while the memory logically belongs to the Fortran result. Never pass a live, mutable NumPy array as an input to a wrapper if another thread or library could modify it during the same call.
- Check `ierr` immediately after every call via `check_err_code`.
- Functions that return more than one value must return a **dictionary** keyed by output name (e.g. `return {'tissue_versatilities': ..., 'tissue_angles_deg': ...}`). A function with a single output value returns it directly.
- **Do not edit `python/tensor_omics/`.** The whole package is generated, docstrings included — they are rendered from the Fortran doc comments. A change belongs in the implementation's source or its annotations.

18 FORD Documentation

FORD (Fortran Documentation Generator) produces HTML documentation directly from structured comments. All public procedures and modules must be documented.

18.1 Comment Syntax

- `!>` — Starts the description of a module, subroutine, or function.
- `!|` — Continues or extends a description (e.g. a second line after `!>`); also used to document an argument when placed **before** the declaration.
- `!!` — Trail comment placed **after** the declaration (on the following line, indented). This is the preferred way to document individual arguments in this codebase.

In summary: `!|` before, `!!` after. Both are valid FORD syntax; the template below uses `!!` consistently.

18.2 Module and Subroutine Template

```
!> Module for computing expression centroids of gene families.
!| Contains the core scientific kernel. C and language wrappers are defined
!| outside the module for compatibility.
module tox_gene_centroids
  use, intrinsic :: iso_fortran_env, only: real64, int32
  implicit none
contains

  !> Computes the element-wise mean for a given set of gene expression vectors.
  !| Returns a single centroid vector of length n_axes.
  pure subroutine mean_vector(expression_vectors, n_axes, n_genes, &
      gene_indices, n_selected_genes, centroid, ierr)
    integer(int32), intent(in) :: n_axes
      !! Number of axes (dimensions)
    integer(int32), intent(in) :: n_genes
      !! Total number of genes in the expression matrix
    real(real64), intent(in) :: expression_vectors(n_axes, n_genes)
      !! Gene expression matrix (n_axes x n_genes)
    integer(int32), intent(in) :: n_selected_genes
```

```
      !! Number of genes to include in the centroid
    integer(int32), intent(in) :: gene_indices(n_selected_genes)
      !! Column indices of selected genes in expression_vectors
    real(real64), intent(out) :: centroid(n_axes)
      !! Output centroid vector (n_axes)
    integer(int32), intent(out) :: ierr
      !! Error code: 0 = success
    ! ... implementation ...
  end subroutine mean_vector

end module tox_gene_centroids
```

Note the use of `!!` (two exclamation marks) to place the documentation comment on the line *after* the declaration. This is the FORD convention used throughout the codebase.

18.3 Generating Documentation

Documentation is generated automatically by the CI/CD workflow on every merge to the main branch. **You do not need to run FORD manually.** However, if you want to preview the output locally before submitting an MR:

```
ford ford.yml         # run from project root
open doc/index.html   # review in browser
```

19 Testing

The project provides a structured test suite framework. See the test directory README for full details on how to add and run tests.

19.1 Where to Test What

- **Algorithm correctness**: Fortran unit tests only. The Fortran test suite is the single source of truth for numerical correctness.
- **Python / R wrappers**: Test only that the function can be called, that the return value is of the correct type and shape, and that reasonable input produces a non-error result.

19.2 Test Naming Convention

Each test file is named `mod_test_<module_name>.{f90|F90|py|R}`. Each test function is named `test_<description>`.

20 Merge Request and Collaboration Conventions

- **Do not merge your own MR.** Always request at least one reviewer and wait for approval before merging.
- **Always assign experienced developers as reviewers.** Add at least Franz or Aaron as a reviewer on every MR. They have deep knowledge of the codebase and can catch issues early.
- **Do not close review threads yourself.** Only the person who opened a thread (the reviewer) should close it after it has been resolved to their satisfaction.
- **Resolve conflicts before requesting review.** Rebase onto the target branch and fix conflicts yourself.

- **Keep MRs focused.** One logical change per MR. Large refactors should be discussed in an issue before starting the MR.
- **Link to the relevant issue** in the MR description.
- **Update FORD documentation** and tests as part of the same MR as the implementation.
- **Check the CI/CD workflow.** After pushing or updating an MR, always verify that the automated workflow (tests, build, documentation generation) passes. If it fails, *fix it yourself* — do not wait to be told. A failing pipeline blocks the merge and means the code is not yet ready for review.
- **Use GitHub, not GitLab.** Since May 2026 the project is developed exclusively on GitHub. Do not open issues, branches, or MRs on the old GitLab repository — those will not be seen or merged.

21 Getting Help: The ask-for-help Channel

The ask-for-help channel is the right place for **any question** — not just programming. Whether it is about scientific context, workflow setup, naming decisions, tooling, or general project conventions, there will always be someone available to help. Do not hesitate to ask.

For *technical problems* (especially compilation errors), do not spend more than 30 minutes debugging alone. Post in the channel with:

1. The exact compiler command and full error message (copy-paste, do not paraphrase).
2. The minimal code snippet that triggers the error.
3. What you have already tried.

Many gfortran errors for `bind(C)`, `pure`, or kind-mismatch issues are non-obvious; the team has likely encountered the same problem before.

22 Code Snippets

Snippets for Fortran, Python, and R are **auto-generated** by the code generator and written to the `snippets/` directory. You should never edit them by hand. Note that `snippets` is not one of the default targets, so an ordinary `python helper/generate_code.py` run leaves them alone — regenerate them explicitly with `python helper/generate_code.py --target snippets` whenever a published procedure is renamed or its signature changes.

22.1 How Snippet Generation Works

The generator reads the same annotated Fortran source that produces the bindings, so a snippet cannot name a procedure that does not exist or carry a signature that has drifted — both come from the parsed source rather than from a header comment a human keeps in step. It emits, per language:

- **Fortran** — a call snippet for each published subroutine, keyed `tox:<name>` or `f42:<name>` by which library the module belongs to.
- **Python and R** — a call snippet for each generated wrapper function.
- `toxdev:*` — development conveniences (parallel loop templates, argument declarations) that are not tied to any one procedure.

22.2 Running It

```
python helper/generate_code.py --target snippets
```

There is nothing to keep in step by hand: no `#>` header to write, no naming rule to observe. Give the procedure a good `!> summary:` line and it becomes the snippet's description.

23 Integrating External Fortran Code from Netlib

Netlib is the canonical archive for legacy scientific Fortran (BLAS, LINPACK, LOESS, etc.). This section explains how to extract, compile, and link Netlib packages so they can be added to `external/`.

23.1 Shell Archives (shar)

Netlib sometimes distributes packages as **Shell Archives** (`.shar`): self-extracting shell scripts that reconstruct source files when executed. Each line of actual source is prefixed with a hyphen; the script uses `sed` to strip those prefixes during extraction and write the real files. This format dates to the 1980s and was also a safety measure against accidental execution of embedded shell commands.

Do not copy-paste the text manually. Save it to a file and run it:

```
wget https://netlib.org/a/loess
sh loess          # extracts README, loessf.f, lowesf.f, lowescp.f, ...
```

Perform extraction in a **temporary directory outside the project**. Once compilation is confirmed working, move the relevant files into `external/`.

23.2 Checking Dependencies

After extraction, read the `README`. Many Netlib packages omit routines they assumed were already present on the target system. LOESS, for example, depends on LINPACK routines (`dqrsl`, `dsvdc`) that are not included in the distributed archive. Missing symbols cause linker errors such as:

```
undefined reference to 'dqrsl'
undefined reference to 'dsvdc'
```

Fetch the missing `.f` files from Netlib and add them to the compilation.

23.3 Compiling Legacy Fortran Sources

Old Fortran (fixed-form, F77) requires special flags:

```
gfortran -O2 -std=legacy -ffixed-line-length-none \
         -fallow-argument-mismatch -fPIC -w \
         -c file1.f file2.f file3.f
```

Flag	Purpose
`-O2`	Standard optimisations.
`-std=legacy`	Accepts F77 constructs no longer valid in modern standards.
`-ffixed-line-length-none`	Removes the 72-character column limit of fixed-form source.
`-fallow-argument-mismatch`	Suppresses fatal errors from implicit-interface argument-type mismatches common in old code.
`-fPIC`	Position-independent code; required when object files will be linked into a shared library (`.so`).
`-w`	Suppresses warnings. Use it only after the initial integration is stable; omit it while diagnosing problems.
`-c`	Compile only (produce `.o` files); do not link.

23.4 Linking

To build a **static library** for inclusion in Tensor-Omics:

```
ar rcs libname.a *.o
```

To build a standalone **executable** for testing:

```
gfortran -o program_name file1.o file2.o file3.o
```

If `undefined reference` errors appear at link time, a dependency is missing. Classic culprits in Netlib packages are additional routines from **BLAS**, **LINPACK**, or utility functions such as `d1mach` — fetch and compile those as well.

24 Quick-Reference Checklist

Use this checklist when reviewing or submitting any Fortran code contribution:

- ☐ File extension is `.F90` (macros used) or `.f90` (no macros).
- ☐ `#include "macros.h"` is present if any `M_*` macro is used.
- ☐ All `real` variables use `real(real64)`; all constants carry `_real64`.
- ☐ Every argument has `intent(in|out|inout)`.
- ☐ Core (helper) routines are declared `pure`.
- ☐ `do concurrent` is used for independent loops.
- ☐ No `GOTO` anywhere.
- ☐ `ierr` is the last (or only output) argument and is initialised with `set_ok(ierr)`.
- ☐ Every `set_err` / allocation failure path returns immediately.
- ☐ Core routines do not allocate; allocation lives in the generated entry point.
- ☐ No unexplained copies of large arrays.
- ☐ C wrappers follow the `_c` / `_expert_c` naming convention.
- ☐ All C wrapper arguments carry `target` and are passed by reference.
- ☐ Null checks use `M_CHECK_IERR_NON_NULL` first, then `M_CHECK_NON_NULL` for all others.
- ☐ No derived types in C-interfaced code.
- ☐ `use safeguard` is present in every module with C wrappers.
- ☐ All utility functions are in `f42_utils` (or re-use existing ones).
- ☐ FORD comments (`!>`, `!|`) are present for all public procedures.
- ☐ Fortran tests cover the new logic.
- ☐ Python/R tests verify call-ability and return type.
- ☐ MR does not self-merge and review threads are not self-closed.